МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут ІКНІ
Кафедра СШІ

ЗВІТ

до лабораторної роботи № 5
з дисципліни: «Інтелектуальний аналіз та візуалізація даних»
на тему: «Географічні дані OpenStreetMap: від тегів до маршруту»

Виконав: ст. гр. ШІ-33 Левченко Д.К.
Прийняла: Торська Роксана

Львів – 2026

Тема роботи:

Географічні дані OpenStreetMap: від тегів до маршруту — відбір історико-культурних об’єктів міста, побудова критерію їх цінності та пішого маршруту від залізничного вокзалу.

Мета роботи:

Навчитися отримувати геодані OpenStreetMap живим запитом і зі збереженого знімка, відбирати об’єкти за тегами, конструювати й обґрунтовувати критерій рейтингу, рахувати віддалі в метричній системі координат пішохідною мережею, розв’язувати задачу комівояжера наближеними й точним методами, будувати просторовий індекс H3, просторове поєднання та інтерактивну карту маршруту.

Індивідуальне завдання (варіант 4):

Місто визначається номером у списку групи: номер 4 відповідає четвертому місту списку — Одесі. Для неї треба відібрати десять найвартісніших історико-культурних об’єктів, скласти піший маршрут від залізничного вокзалу і назад, а також розв’язати ту саму задачу для тридцяти об’єктів як базу порівняння для лабораторної роботи № 8 (табл. 1).

Таблиця 1 — Індивідуальне завдання

Параметр	Значення
Місто	Одеса, Україна; межа міста — 163,3 км²
Джерело даних	OpenStreetMap: Nominatim і Overpass API через osmnx 2.1.1; дані © OpenStreetMap contributors, ліцензія ODbL
Шари	межа міста; heritage (historic і tourism); вокзали; заклади (cafe, restaurant, drinking_water, toilets); пішохідна мережа walk
Старт і фініш	залізничний вокзал Одеса-Головна
Мала задача	вокзал і 10 об’єктів, точний розв’язок Held–Karp
Велика задача	вокзал і 30 об’єктів, наближені методи; база порівняння для ЛР8
Результати	ЛР5_Левченко.ipynb, lr5_результати.json, lr5_маршрут.html, знімок даних у теці osm_одеса

ЗАВДАННЯ

1. Отримати дані міста двома способами — живим запитом і зі знімка — та порівняти їх за часом і обсягом переданих даних.

2. Відібрати кандидатів у пам’ятки за об’єднанням тегів historic і tourism та назвати, що й чому відкинуто.
3. Побудувати критерій «найкращі» щонайменше з трьох сигналів, порівняти його з рейтингом за площею та знайти дублікати.
4. Обрати вокзал і прив’язати точки до вузлів пішохідного графа з перевіркою злипання.
5. Побудувати матриці віддалей прямою і дорогою та обчислити коефіцієнт обходу.
6. Розв’язати задачу комівояжера найближчим сусідом, 2-opt і Held–Karp для 10 точок і наближено для 30 точок.
7. Розкласти кандидатів по комірках H3 рівня 9 і побудувати картограму щільності.
8. Знайти найближчий заклад до кожної точки маршруту просторовим поєднанням.
9. Побудувати карту, на якій видно маршрут по ребрах графа і прямі відрізки між точками.
10. Зберегти результати для лабораторної роботи № 8 і відповісти на сім питань звіту.

ТЕОРЕТИЧНІ ВІДОМОСТІ

OpenStreetMap зберігає три типи об’єктів: вузол (точка з координатами), шлях (впорядкований список вузлів — вулиця, контур будівлі) і відношення (група об’єктів із ролями). Сенс об’єкта задають теги «ключ = значення». Тега «визначна пам’ятка» в OSM немає, тому кандидатів беруть з об’єднання двох сімейств: historic (castle, church, monument, memorial, ruins тощо) і tourism (museum, gallery, attraction, artwork, viewpoint). Один і той самий вид об’єкта трапляється в різних типах геометрії: музей у власній будівлі — полігон, а пам’ятник — точка.

Дані OSM приходять у EPSG:4326 — у градусах широти й довготи, тому довжини й площі в них безглузді. Усі метричні обчислення виконано в EPSG:32635 (UTM, зона 35N). Порядок координат різний: shapely.Point приймає (довгота, широта), а folium — (широта, довгота).

Маршрут іде ребрами пішохідного графа (network_type="walk"): кожну точку «садять» на найближчий вузол. На автомобільній мережі кілька пам’яток старого міста злипаються на одному вузлі, і віддаль між ними стає нульовою.

Відношення віддалі дорогою до прямої — коефіцієнт обходу — для пішохідної мережі міста зазвичай 1,2–1,6.

Задача комівояжера — знайти найкоротший замкнений маршрут через усі точки рівно по одному разу. Для неї немає алгоритму, який швидко дає точну відповідь у загальному випадку, тому порівнюють три методи (табл. 2).

Таблиця 2 — Методи розв'язання задачі комівояжера

Метод	Що робить	Час	Що гарантує
Найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$	нічого
2-opt	розвертає ділянки маршруту, поки довжина зменшується	$O(n^2)$ за прохід	локальний мінімум
Held–Karp	динамічне програмування за підмножинами	$O(2^n \cdot n^2)$	точний оптимум

Таблиця Held–Karp має $2^{n-1} \cdot (n - 1)$ клітин, тож при кожних п'яти додаткових точках потреба в пам'яті зростає приблизно в тридцять разів. H3 — ієрархічна сітка шестикутників; на рівні 9 середнє ребро комірки близько 200 м. Просторове поєднання sjoin_nearest рахує відстань в одиницях системи координат, тому обидва шари спершу переводять у метричну систему.

ХІД РОБОТИ

Етап 1. Дані двома способами

Функцію fetch доповнено гілкою живого запиту: перед викликом запам'ятовується розмір кешу osmnx і час, результат проходить через clean() і зберігається у GeoParquet, а в таблицю COST записується спосіб, кількість записів, передані й збережені мегабайти та час. Щоб порівняння лишилось видимим і на повторному запуску, показники першого (живого) запуску зберігаються у файлі cost_live.json теки знімка.

```
def fetch(name, getter):
    """Повертає шар. Якщо файл уже збережено — читає з диска."""
    path = os.path.join(OUT, name + '.parquet')
    if os.path.exists(path) and os.path.getsize(path) > 0:
        t0 = time.time()
        g = gpd.read_parquet(path)
        COST[name] = {'спосіб': 'знімок', 'записів': len(g),
                      'передано_МБ': 0.0,
                      'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                      'секунд': round(time.time() - t0, 3)}
        return g
```

```
    t0, b0 = time.time(), cache_bytes()
    g = clean(getter())
    g.to_parquet(path, index=False)
    COST[name] = {'спосіб': 'живий запит', 'записів': len(g),
                  'передано_МБ': round((cache_bytes() - b0) / 1e6, 3),
                  'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                  'секунд': round(time.time() - t0, 1)}
    return gpd.read_parquet(path)
```

```
LIVE_P = os.path.join(OUT, 'cost_live.json')
live = {k: v for k, v in COST.items() if v['спосіб'] == 'живий запит'}
if live:
    saved = json.loads(open(LIVE_P, encoding='utf-8').read()) if
os.path.exists(LIVE_P) else {}
    saved.update(live)
    with open(LIVE_P, 'w', encoding='utf-8') as f:
        json.dump(saved, f, ensure_ascii=False, indent=1)

cost = pd.DataFrame(COST).T
display(cost)

first = pd.DataFrame(json.loads(open(LIVE_P, encoding='utf-8').read())).T
speed = pd.DataFrame({
    'живий запит, с': first['секунд'].astype(float),
    'знімок, с': cost['секунд'].astype(float),
    'передано мережею тоді, МБ': first['передано_МБ'].astype(float),
    'передано зараз, МБ': cost['передано_МБ'].astype(float),
})
speed['знімок швидший, разів'] = (speed['живий запит, с'] / speed['знімок,
с'].clip(lower=0.001)).round(0)
display(speed)

SPEEDUP = speed['живий запит, с'].sum() / max(speed['знімок, с'].sum(), 0.001)
MB_SAVED = speed['передано мережею тоді, МБ'].sum()
print('живий запит усіх шарів: %.0f с; читання знімка: %.2f с; знімок швидший у %.0f
разів'
      % (speed['живий запит, с'].sum(), speed['знімок, с'].sum(), SPEEDUP))
print('при повторному запуску мережею не передано %.1f МБ' % MB_SAVED)
```

Результат виконання:

```
живий запит усіх шарів: 310 с; читання знімка: 0.37 с; знімок швидший у 830 разів
при повторному запуску мережею не передано 14.5 МБ
```

Таблиця 3 — Вартість двох способів доступу до даних

Шар	Живий запит, с	Знімок, с	Передано мережею, МБ	Знімок швидший, разів
boundary	2,3	0,135	0,050	17
heritage	55,6	0,065	0,860	855
stations	69,5	0,012	0,020	5 792
amenities	92,9	0,023	0,416	4 039
мережа walk	89,3	0,138	13,196	647

Перший, живий, запуск усіх шарів тривав 310 с, а повторне читання знімка — 0,37 с, тобто знімок швидший приблизно в 830 разів. Повторний запуск не

передав мережею 14,5 МБ, з них 13,2 МБ — пішохідна мережа (28 641 вузол і 78 130 ребер). На другому запуску всі шари мають спосіб «знімок» і нульовий трафік — це контрольна ознака, що fetch кешує.

Етап 2. Кандидати в пам’ятки

```
tags_ok = heritage['historic'].notna() | heritage['tourism'].isin(TOURISM_OK)
has_name = heritage['name'].notna() & (heritage['name'].str.strip() != '')
cand = heritage[tags_ok & has_name].reset_index(drop=True)
dropped = heritage[tags_ok & ~has_name]


print('усього у шарі heritage :', len(heritage))
print('підходять за тегами    :', int(tags_ok.sum()))
print('з них мають name       :', len(cand))
print('відкинуто без name     :', len(dropped))
print()
print('що саме відкинуто (historic):')
print(dropped['historic'].value_counts().head(8).to_string())
print()
print('що саме відкинуто (tourism):')
print(dropped['tourism'].value_counts().head(8).to_string())
```

Результат виконання:

```
усього у шарі heritage : 895
підходять за тегами    : 895
з них мають name       : 547
відкинуто без name     : 348


що саме відкинуто (historic):
historic
heritage         163
memorial          74
yes                3
fort               3
castle             2
bicycle            2
monument           2
wayside_cross      1

що саме відкинуто (tourism):
tourism
artwork       56
attraction    37
viewpoint      6
```

Усі 895 об’єктів шару підходять за об’єднанням тегів, але 348 з них (39 %) не мають назви, і їх відкинуто: маршрут не може вести до «якоїсь історичної будівлі». Найбільше серед відкинутих historic=heritage (163 — пам’ятки архітектури без назви) і historic=memorial (74). Лишилось 547 кандидатів; серед них 343 точки, 199 полігонів і 5 ліній — тобто фільтр не втратив пам’ятників і оглядових точок.

Етап 3. Критерій «найкращі»

Стовпця «цінність» у даних немає, тому його сконструйовано з п’яти сигналів OSM, кожен з яких зведено до спільної шкали часток від 0 до 1 через rank(pct=True). Найбільші ваги мають сигнали зовнішнього визнання: наявність статті у Вікіпедії або Wikidata (0,35) і кількість перекладів назви (0,30). Музей чи галерея отримують 0,15, бо туди можна зайти; заповнений historic — 0,10; площа як найслабший сигнал — 0,10.

```
import re
LANG_COLS = [c for c in cand.columns if re.fullmatch(r'name:[a-z]{2,3}', c)]
cand['wiki'] = (cand['wikidata'].notna() | cand['wikipedia'].notna()).astype(int)
cand['n_lang'] = cand[LANG_COLS].notna().sum(axis=1)
cand['museum'] = cand['tourism'].isin(['museum', 'gallery']).astype(int)
cand['hist'] = cand['historic'].notna().astype(int)


WEIGHTS = {'wiki': 0.35, 'n_lang': 0.30, 'museum': 0.15, 'hist': 0.10, 'area_m2':
0.10}
cand['score'] = sum(w * cand[c].rank(pct=True) for c, w in WEIGHTS.items())


print('кандидатів   :', len(cand))
print('мають wiki   :', int(cand['wiki'].sum()))
print('мають переклади назви: %d, максимум мов: %d' % ((cand['n_lang'] > 0).sum(),
cand['n_lang'].max()))
print('музеї та галереї: %d, historic заповнений: %d' % (cand['museum'].sum(),
cand['hist'].sum()))
display(cand.nlargest(10, 'score')[['name', 'historic', 'tourism', 'wiki', 'n_lang',
'area_m2', 'score']].round(3))
```

Результат виконання:

```
кандидатів   : 547
мають wiki   : 47
мають переклади назви: 533, максимум мов: 9
музеї та галереї: 47, historic заповнений: 356
```

```
def keys(df):
    return set(df['element'].astype(str) + '/' + df['id'].astype(str))


top_score = cand.nlargest(N_LARGE, 'score').reset_index(drop=True)
top10 = top_score.head(N_SMALL)
top_naive = cand.nlargest(N_SMALL, 'area_m2').reset_index(drop=True)
overlap = len(keys(top10) & keys(top_naive))
print('перетин: %d з %d' % (overlap, N_SMALL))
display(pd.DataFrame({'за вашим критерієм': list(top10['name']),
                      'за площею': list(top_naive['name'])}))
```

Результат виконання:

```
перетин: 1 з 10
```

Таблиця 4 — Перша десятка за двома критеріями

№	За моїм критерієм	За площею

№	За моїм критерієм	За площею
1	Літературний музей	Свято-Успенський патріарший чоловічий монастир
2	Археологічний музей	Ангари
3	Потьомкінські сходи	Гарматні дворики
4	Одеський залізничний вокзал	Одеський художній музей
5	Одеський національний художній музей	Одеський академічний театр музичної комедії
6	Воронцовський палац	Одеський залізничний вокзал
7	Синагога Ор Самеах	Одеський національний академічний театр опери та балету
8	Прибутковий будинок Асвадурова	Ангари депо
9	Римсько-католицький костьол Успіння Пресвятої Богородиці Діви Марії	Пасаж Менделевичів
10	Шахський палац	Тюремний замок

У двох десятках збігся лише один об’єкт — залізничний вокзал. За площею в рейтинг потрапили ангари, депо й тюремний замок: великим буває і склад. Обидва списки отримано з тих самих даних без жодної помилки, тож «найкращі» — властивість критерію, який обирає аналітик, а не властивість даних.

Далі перевірено дублікати: функцію close_pairs спершу випробувано на двох точках за 14 м, а потім знайдено пари в топ-30, ближчі за 60 м. З кожної пари лишається об’єкт з вищим рейтингом. Додатково прибрано два пам’ятники, які в OSM позначені як демонтовані, і знайдено об’єкти з однаковим wikidata.

```
control = gpd.GeoDataFrame(geometry=gpd.points_from_xy([30.7300, 30.7301], [46.4800,
46.4801]), crs=4326)
print('контроль функції на двох точках за ~14 м:', [(i, j, round(float(d), 1)) for
i, j, d in close_pairs(control)])
print()


pairs = close_pairs(top_score)
print('пар ближче 60 м у топ-%d: %d' % (N_LARGE, len(pairs)))
for i, j, d in pairs:
    print('  %5.1f м | %s  <->  %s' % (d, top_score['name'][i],
top_score['name'][j]))


gone = cand['name'].str.contains('демонтован', case=False)
print()
print('у назві позначені як демонтовані (на місці їх уже немає):', ',
'.join(cand.loc[gone, 'name']))
same_wd = cand[cand['wikidata'].notna() & cand['wikidata'].duplicated(keep=False)]
print('кандидати з однаковим wikidata (той самий об\u2019єкт двічі):')
print(same_wd[['element', 'name', 'historic', 'tourism',
'wikidata']].sort_values('wikidata').to_string())


def dedupe(pool):
    removed = []
    while True:
```

```
        found = close_pairs(pool.head(N_LARGE))
        if not found:
            return pool, removed
        i, j, d = found[0]
        removed.append((pool['name'][j], pool['name'][i], d))
        pool = pool.drop(index=j).reset_index(drop=True)

pool = cand[~gone].sort_values('score', ascending=False).reset_index(drop=True)
pool, removed = dedupe(pool)

top_score = pool.head(N_LARGE).reset_index(drop=True)
top10 = top_score.head(N_SMALL)
overlap = len(keys(top10) & keys(top_naive))
print()
print('рішення: з кожної пари лишаємо об’єкт з вищим рейтингом, другий прибираємо')
for a, b, d in removed:
    print('  прибрано «%s» — %.0f м від «%s»%s' % (a, d, b, ' (лишилась інша точка з
тією самою назвою)' if a in set(pool['name']) else ''))
print('після чистки пар ближче 60 м у топ-%d: %d' % (N_LARGE,
len(close_pairs(top_score))))
print('перетин рейтингів після чистки: %d з %d' % (overlap, N_SMALL))
display(top10[['name', 'historic', 'tourism', 'wiki', 'n_lang', 'score']].round(3))
```

Результат виконання:

```
контроль функції на двох точках за ~14 м: [(0, 1, 13.5)]


пар ближче 60 м у топ-30: 4
   21.2 м | Одеський національний художній музей  <->  Одеський художній музей
   22.4 м | Пам'ятник засновникам Одеси (демонтований)  <->  Одеський художній музей
   25.8 м | Одеський національний художній музей  <->  Пам'ятник засновникам Одеси
(демонтований)
   54.9 м | Літературний музей  <->  Археологічний музей


у назві позначені як демонтовані (на місці їх уже немає): Пам'ятник засновникам
Одеси (демонтований), Олександр Суворов (демонтований)
кандидати з однаковим wikidata (той самий об’єкт двічі):
      element                                  name  historic     tourism  wikidata
4        node                    Потьомкінські сходи      <NA>   viewpoint  Q1413595
348  relation                    Потьомкінські сходи       yes  attraction  Q1413595
285      node                                 ЗіС-3  memorial        <NA>   Q266837
286      node                                 ЗіС-3  memorial        <NA>   Q266837
515       way  Одеський національний художній музей      <NA>      museum  Q4331480
516       way               Одеський художній музей      <NA>     gallery  Q4331480


рішення: з кожної пари лишаємо об’єкт з вищим рейтингом, другий прибираємо
  прибрано «Одеський художній музей» — 21 м від «Одеський національний художній
музей»
  прибрано «Будинок Лібмана» — 42 м від «Будинок О. П. Руссова»
  прибрано «Археологічний музей» — 55 м від «Літературний музей»
  прибрано «Потьомкінські сходи» — 14 м від «Пам'ятник де Рішельє» (лишилась інша
точка з тією самою назвою)
після чистки пар ближче 60 м у топ-30: 0
перетин рейтингів після чистки: 1 з 10
```

Знайдено три різні типи дублікатів: художній музей записано двічі (museum і gallery з тим самим wikidata), Потьомкінські сходи — відношенням і окремою точкою-оглядовим майданчиком, а «Пам’ятник засновникам Одеси» позначено в назві як демонтований — маршрут вів би до неіснуючої пам’ятки.

Етап 4. Вокзал і прив’язка до графа

```
main = stations[stations['name'].fillna('').str.contains('Одеса-Головна')]
if (main['railway'] == 'station').any():
    main = main[main['railway'] == 'station']
station = main.head(1)
print('старт:', station['name'].iloc[0])


sp = station.to_crs(32635).geometry.representative_point().iloc[0]
d_start = pool.to_crs(32635).geometry.representative_point().distance(sp)
near_start = d_start < 100
print('об\u2019єкти рейтингу ближче 100 м до старту:',
      ', '.join('%s (%.0f м)' % (n, d) for n, d in zip(pool.loc[near_start, 'name'],
d_start[near_start])) or 'немає')
near_names = ['%s — %.0f м' % (n, d) for n, d in zip(pool.loc[near_start, 'name'],
d_start[near_start])]
pool, removed_more = dedupe(pool[~near_start].reset_index(drop=True))
removed += removed_more
top_score = pool.head(N_LARGE).reset_index(drop=True)
top10 = top_score.head(N_SMALL)
overlap = len(keys(top10) & keys(top_naive))
print('пар ближче 60 м у топ-%d: %d; перетин рейтингів: %d з %d' % (N_LARGE,
len(close_pairs(top_score)), overlap, N_SMALL))
print('десятка для маршруту:', ', '.join(top10['name']))
```

Результат виконання:

```
старт: Одеса-Головна
об’єкти рейтингу ближче 100 м до старту: Одеський залізничний вокзал (71 м), EMS
Ukraine (65 м)
пар ближче 60 м у топ-30: 0; перетин рейтингів: 1 з 10
десятка для маршруту: Літературний музей, Потьомкінські сходи, Одеський національний
художній музей, Воронцовський палац, Синагога Ор Самеах, Прибутковий будинок
Асвадурова, Римсько-католицький костьол Успіння Пресвятої Богородиці Діви Марії,
Шахський палац, Пам'ятник де Рішельє, Одеський національний академічний театр опери
та балету
```

```
def check_snap(points, ids, dist, label):
    k, uniq = len(ids), len(set(ids))
    worst = int(np.argmax(dist))
    print('%s: точок %d, різних вузлів %d' % (label, k, uniq))
    print('   точка -> вузол: медіана %.1f м, максимум %.1f м («%s»)'
          % (np.median(dist), dist[worst], points['name'].iloc[worst]))
    if uniq < k:
        same = collections.defaultdict(list)
        for name, node in zip(points['name'], ids):
            same[node].append(name)
        for node, names in same.items():
            if len(names) > 1:
                print('   злиплись на вузлі %s: %s' % (node, ' | '.join(names)))
    return uniq == k
```

Результат виконання:

```
мала задача: точок 11, різних вузлів 11
   точка -> вузол: медіана 31.4 м, максимум 40.9 м («Воронцовський палац»)
```

Стартом обрано станцію Одеса-Головна. Будівля «Одеський залізничний вокзал» за 71 м від неї є водночас пам’яткою з першої десятки, тож у маршруті вона зайняла б одне з десяти місць безплатно. Тому об’єкти ближче 100 м до

старту прибрано з рейтингу; серед них виявився будинок «EMS Ukraine» — пам’ятка архітектури, якій у OSM дали назву кур’єрської служби. Усі 11 точок сіли на 11 різних вузлів, медіана відстані «точка → вузол» — 31,4 м, максимум — 40,9 м.

Етап 5. Матриця віддалей

```
def straight_matrix(points):
    p = points.to_crs(32635).geometry
    XY = np.c_[p.x.values, p.y.values]
    return np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))


def detour_factor(D, E):
    off = ~np.eye(len(D), dtype=bool) & np.isfinite(D) & (E > 0)
    return float(np.median(D[off] / E[off]))
```

Результат виконання:

```
матриця за 0.356 с
недосяжних пар    : 0
асиметрія         : 0 м
коефіцієнт обходу : 1.20
```

Недосяжних пар немає, асиметрія матриці нульова, бо на пішохідній мережі немає односторонніх обмежень. Коефіцієнт обходу 1,20 для 10 точок і 1,26 для 30 — у типовому діапазоні, для малої задачі на нижній межі: історичний центр Одеси — прямокутна сітка кварталів, і пам’ятки десятки стоять уздовж тих самих вулиць.

Етап 6. Задача комівояжера

```
def nearest_neighbour(D):
    """Маршрут 0 -> ... -> 0. Щоразу йдемо в найближчу невідвідану точку."""
    tour, left = [0], set(range(1, len(D)))
    while left:
        cur = tour[-1]
        nxt = min(sorted(left), key=lambda j: D[cur, j])
        tour.append(nxt)
        left.remove(nxt)
    return tour + [0]


def two_opt(D, tour):
    """Беремо маршрут і розвертаємо ділянки, поки довжина зменшується."""
    best, best_len = list(tour), tour_length(D, tour)
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best) - 1):
                new = best[:i] + best[i:j + 1][::-1] + best[j + 1:]
                L = tour_length(D, new)
                if L < best_len - 1e-9:
                    best, best_len, improved = new, L, True
    return best
```

Результат виконання:

```
найближчий сусід: структура маршруту правильна (11 точок)
2-opt: структура маршруту правильна (11 точок)
Held-Karp: структура маршруту правильна (11 точок)


усі перевірки пройдено


оптимальний порядок:
   0. СТАРТ: Одеса-Головна
   1. Прибутковий будинок Асвадурова
   2. Літературний музей
   3. Пам'ятник де Рішельє
   4. Потьомкінські сходи
   5. Воронцовський палац
   6. Шахський палац
   7. Одеський національний художній музей
   8. Одеський національний академічний театр опери та балету
   9. Римсько-католицький костьол Успіння Пресвятої Богородиці Діви Марії
  10. Синагога Ор Самеах
  11. СТАРТ: Одеса-Головна


довжина: 8.81 км
```

Таблиця 5 — Порівняння методів на малій задачі (вокзал і 10 об’єктів)

Метод	Довжина, м	Гірше оптимуму, %
Найближчий сусід	9 279	5,35
2-opt	8 807	0,00
Held–Karp (точний)	8 807	0,00

Найближчий сусід на 5,35 % гірший за оптимум, а 2-opt, запущений з його маршруту, знайшов саме оптимальний маршрут довжиною 8,81 км. Самоперевірки підтвердили, що кожен маршрут починається й закінчується на вокзалі та відвідує кожну точку рівно один раз.

```
growth = pd.DataFrame({'точок': [11, 16, 21, 26, 31]})
growth['клітин'] = [2 ** (n - 1) * (n - 1) for n in growth['точок']]
growth['пам’ять, ГБ'] = growth['клітин'] * 8 / 1e9
growth['операцій'] = [2 ** (n - 1) * (n - 1) ** 2 for n in growth['точок']]
display(pd.DataFrame({
    'точок': growth['точок'],
    'клітин у таблиці': growth['клітин'].map('{:.1e}'.format),
    'пам’яті, ГБ': growth['пам’ять, ГБ'].map('{:.4g}'.format),
    'операцій': growth['операцій'].map('{:.1e}'.format),
}).set_index('точок'))
GB31 = float(growth['пам’ять, ГБ'].iloc[-1])
print('31 точка: %.1e клітин, %.0f ГБ лише на таблицю відстаней' %
(growth['клітин'].iloc[-1], GB31))
```

Результат виконання:

```
31 точка: 3.2e+10 клітин, 258 ГБ лише на таблицю відстаней
```

Таблиця 6 — Зростання таблиці Held–Karp (8 байтів на клітину)

Точок	Клітин у таблиці	Пам’яті	Операцій
11	$1{,}0\cdot10^{4}$	0,08 МБ	$1{,}0\cdot10^{5}$
16	$4{,}9\cdot10^{5}$	3,9 МБ	$7{,}4\cdot10^{6}$
21	$2{,}1\cdot10^{7}$	0,17 ГБ	$4{,}2\cdot10^{8}$
26	$8{,}4\cdot10^{8}$	6,7 ГБ	$2{,}1\cdot10^{10}$
31	$3{,}2\cdot10^{10}$	258 ГБ	$9{,}7\cdot10^{11}$

Результат виконання:

```
велика задача: точок 31, різних вузлів 31
   точка -> вузол: медіана 22.5 м, максимум 46.6 м («Свято-Успенський кафедральний
собор УПЦ МП»)
коефіцієнт обходу: 1.26
2-opt на 30 точках: структура маршруту правильна (31 точок)


найближчий сусід :    28561 м
2-opt            :    24021 м  за 0.006 с  (коротше на 15.9%)


ЦЕ — база порівняння для лабораторної роботи № 8.
```

Для 31 точки таблиця потребує 258 ГБ пам’яті, тож точний розв’язок неможливий через брак пам’яті, а не часу. На великій задачі найближчий сусід дав 28,56 км, а 2-opt — 24,02 км (на 15,9 % коротше) за 0,006 с; ці числа стануть базою порівняння для генетичного й мурашиного алгоритмів у лабораторній роботі № 8.

Етап 7. Індексація H3

```
rp = cand.geometry.representative_point()
cand['h3'] = [h3.latlng_to_cell(p.y, p.x, H3_RES) for p in rp]
dens = cand['h3'].value_counts()
hexes = gpd.GeoDataFrame({'h3': dens.index, 'кількість': dens.values},
                         geometry=[Polygon([(lng, lat) for lat, lng in
h3.cell_to_boundary(c)])
                                   for c in dens.index],
                         crs='EPSG:4326')
HALF = int((dens.cumsum() < len(cand) / 2).sum() + 1)
print('комірок із пам’ятками: %d; половина з %d пам’яток лежить у %d комірках
(%.1f%% комірок)'
      % (len(dens), len(cand), HALF, 100 * HALF / len(dens)))


fig, axes = plt.subplots(1, 2, figsize=(15, 7.5))
for ax, zoom in zip(axes, [False, True]):
    boundary.boundary.plot(ax=ax, color='black', linewidth=0.8)
    hexes.plot(ax=ax, column='кількість', cmap='YlOrRd', legend=not zoom,
               edgecolor='grey', linewidth=0.2)
    pts30.plot(ax=ax, color='#1f77b4', markersize=18, label='точки великої задачі')
    pts30.iloc[[0]].plot(ax=ax, color='black', marker='*', markersize=160,
label='вокзал')
    if zoom:
        x0, y0, x1, y1 = pts30.total_bounds
        ax.set_xlim(x0 - 0.006, x1 + 0.006)
        ax.set_ylim(y0 - 0.004, y1 + 0.004)
    ax.set_aspect(1 / np.cos(np.radians(46.47)))
```

```
    ax.set_xlabel('довгота')
    ax.set_ylabel('широта')
axes[0].set_title('Щільність пам’яток у комірках H3 рівня 9 (усе місто)')
axes[1].set_title('Центр міста: комірки і точки маршруту')
axes[1].legend(loc='lower left')
plt.tight_layout()
plt.show()
```

Результат виконання:

```
середнє ребро комірки: 201 м
комірок із пам’ятками: 219; половина з 547 пам’яток лежить у 34 комірках (15.5% комірок)
```

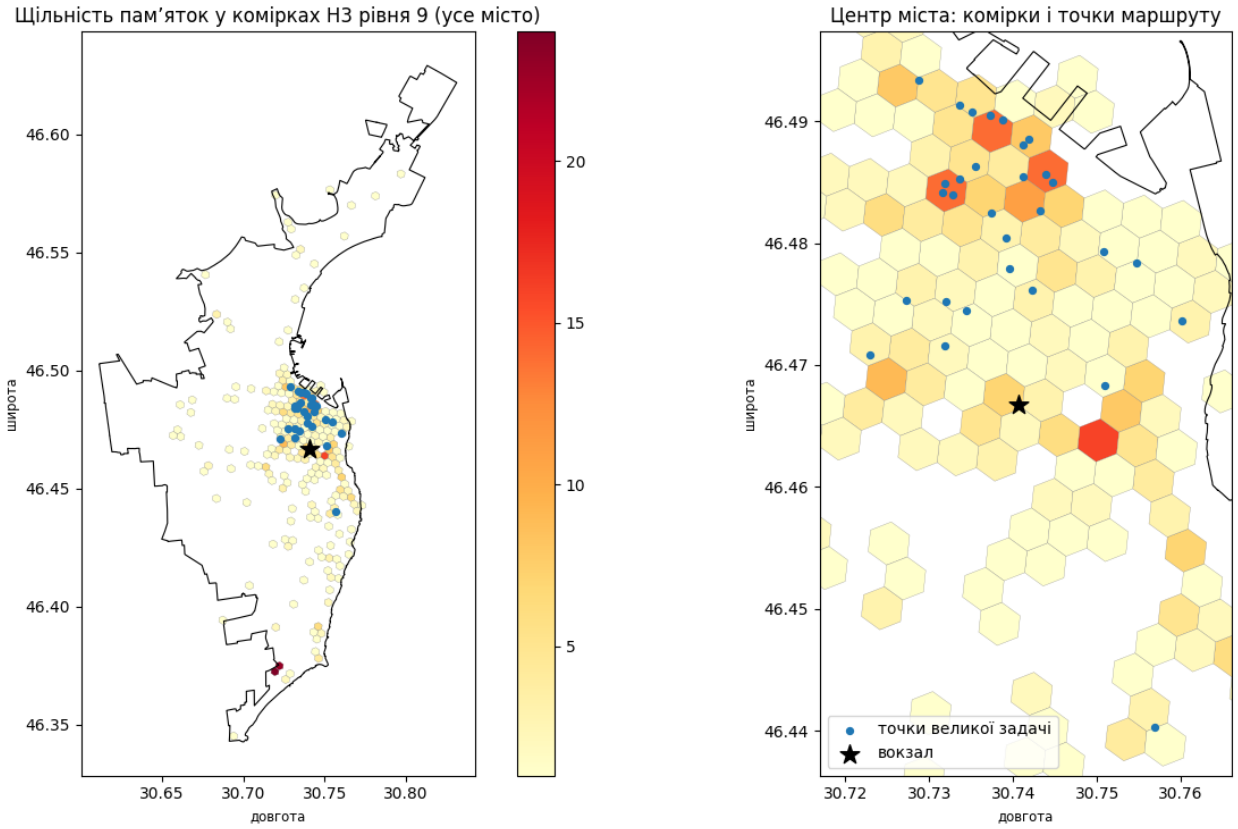


Рисунок 1 — Щільність пам’яток у комірках H3 рівня 9 і точки великої задачі

Середнє ребро комірки рівня 9 — 201 м, що збігається з таблицею H3. Пам’ятки розподілено дуже нерівномірно: 547 кандидатів займають 219 комірок, але половина з них лежить лише в 34 комірках (15,5 %), переважно в історичному центрі. Тому маршрут центром короткий, а будь-який віддалений об’єкт різко його подовжує.

Етап 8. Просторове поєднання

```
am = amenities[['amenity', 'name', 'geometry']].copy()
am['name'] = am['name'].fillna('(без назви)')
route10 = pts10.iloc[ex10[:-1]].reset_index(drop=True)
route10['крок'] = range(len(route10))
```

```
near = gpd.sjoin_nearest(route10.to_crs(32635), am.to_crs(32635), how='left',
distance_col='відстань_м')
near = near[~near.index.duplicated()]
display(near[['крок', 'name_left', 'amenity', 'name_right', 'відстань_м']]
        .rename(columns={'name_left': 'точка маршруту', 'amenity': 'тип закладу',
                         'name_right': 'найближчий заклад'}).round(1))
print('медіана відстані до найближчого закладу: %.0f м' %
near['відстань_м'].median())
```

Результат виконання:

```
медіана відстані до найближчого закладу: 49 м
```

Таблиця 7 — Найближчий заклад до кожної точки маршруту

Крок	Точка маршруту	Заклад	Відстань, м
0	СТАРТ: Одеса-Головна	кафе «Варенична»	42,1
1	Прибутковий будинок Асвадурова	кафе «Дядя Миша»	96,6
2	Літературний музей	кафе «Omega Three»	106,5
3	Пам’ятник де Рішельє	кафе «Франс.уа»	49,3
4	Потьомкінські сходи	туалет	57,5
5	Воронцовський палац	туалет	47,3
6	Шахський палац	туалет	119,3
7	Одеський національний художній музей	туалет	23,2
8	Одеський національний академічний театр опери та балету	ресторан «Щастя»	78,3
9	Римсько-католицький костьол Успіння Пресвятої Богородиці Діви Марії	ресторан «Хаджибей»	49,0
10	Синагога Ор Самеах	ресторан «Хеврон»	24,7

Обидва шари перед поєднанням переведено в EPSG:32635, тому відстані — у метрах. Медіана відстані до найближчого закладу — 49 м: уздовж маршруту центром кав’ярні й туалети трапляються на кожному кроці.

Етап 9. Карта маршруту

```
road10 = route_coords(ex10, ids10)
straight10 = [(float(pts10.geometry.y.iloc[i]), float(pts10.geometry.x.iloc[i])) for
i in ex10]


fmap = folium.Map(location=[np.mean([c[0] for c in straight10]), np.mean([c[1] for c
in straight10])],
                  zoom_start=15, tiles=BASEMAP.build_url(),
                  attr=BASEMAP.html_attribution + ' | дані: &copy; OpenStreetMap
contributors, ODbL')
folium.PolyLine(road10, color='#d62728', weight=5, opacity=0.85,
                tooltip='маршрут по ребрах графа: %.2f км' % (L_opt /
1000)).add_to(fmap)
folium.PolyLine(straight10, color='#1f77b4', weight=2, dash_array='8 6',
                tooltip='прямі відрізки: %.2f км' % (tour_length(E10, ex10) /
1000)).add_to(fmap)
for step, i in enumerate(ex10[:-1]):
    folium.Marker(straight10[step], tooltip='%d. %s' % (step,
pts10['name'].iloc[i]),
                  icon=folium.DivIcon(html='<div
style="background:#222;color:#fff;border-radius:50%%;'
```

```
                                                'width:22px;height:22px;text-
align:center;line-height:22px;'
                                                'font:bold 12px sans-serif">%d</div>' %
step)).add_to(fmap)
fmap.save('lr5_маршрут.html')
print('карту збережено: lr5_маршрут.html')
print('маршрут по ребрах: %.2f км, по прямих відрізках: %.2f км (у %.2f раза довше)'
      % (L_opt / 1000, tour_length(E10, ex10) / 1000, L_opt / tour_length(E10,
ex10)))
fmap
```

Результат виконання:

```
карту збережено: lr5_маршрут.html
маршрут по ребрах: 8.81 км, по прямих відрізках: 7.13 км (у 1.23 раза довше)
```

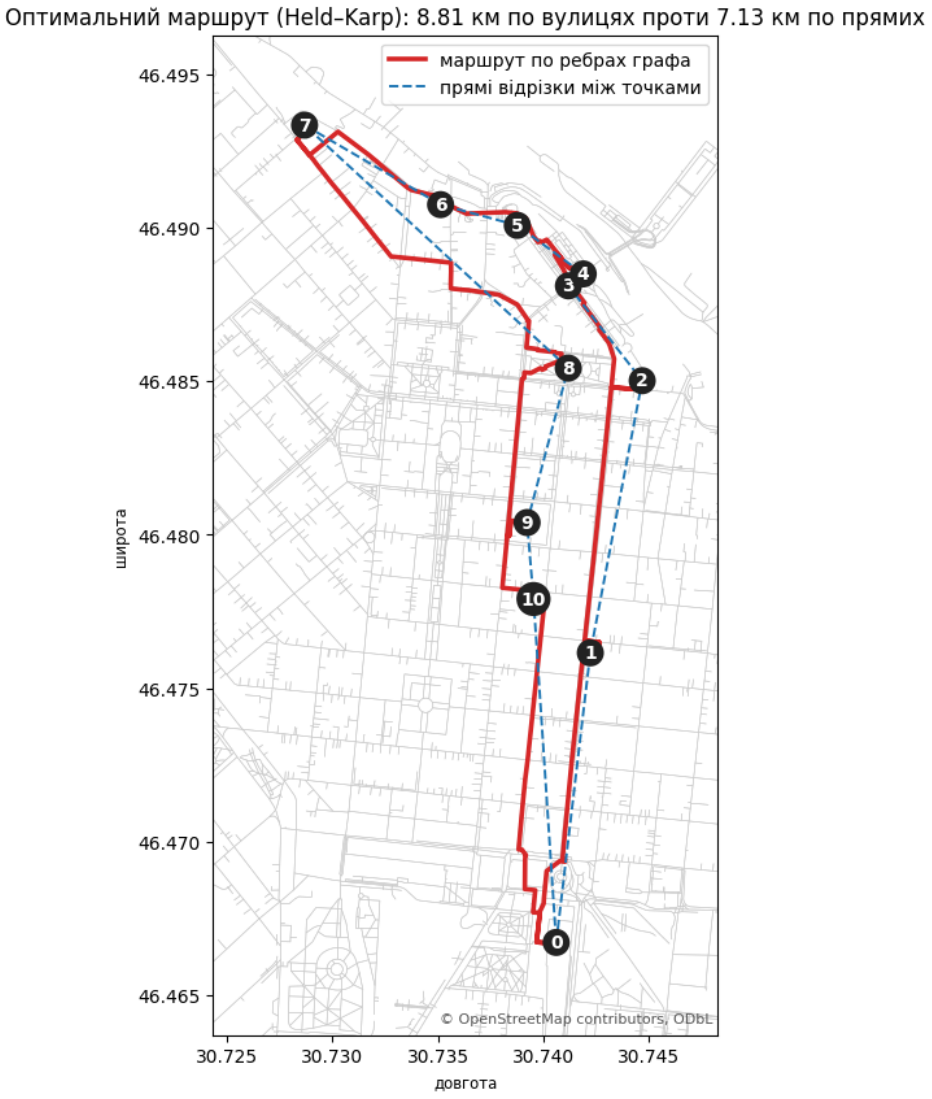


Рисунок 2 — Оптимальний маршрут по вулицях (червоний) і прямі відрізки між точками (синій)

Маршрут по ребрах графа огинає квартали і має довжину 8,81 км, тоді як прямі відрізки між тими самими точками — 7,13 км; різницю видно оком.

Інтерактивну карту з підкладкою Esri.WorldGrayCanvas збережено у файл lr5_маршрут.html, а в атрибуції карти зазначено «© OpenStreetMap contributors, ODbL».

Етап 10. Результати для лабораторної роботи № 8

Файл lr5_результати.json (16,4 КБ) містить студента, місто, результати малої задачі (оптимум 8 807 м, оптимальний порядок, довжини найближчого сусіда й 2-opt) і великої задачі: довжини 28 561 м і 24 021 м, час 2-opt, назви та координати 31 точки і повну матрицю віддалей дорогою. Знімок даних у теці osm_одеса робить результат відтворюваним: на тих самих даних зошит дає ті самі числа.

Етап 11. Відповіді на питання звіту

1. У шарі heritage 895 об’єктів; усі вони підходять за об’єднанням historic або tourism з переліку museum, attraction, gallery, artwork, viewpoint. Без тегу name відкинуто 348 — 39% усіх кандидатів, найбільше historic=heritage (163) і historic=memorial (74). Залишилось 547 іменованих кандидатів. Без пояснень я б такий фільтр замовнику не віддав: він мовчки прибирає 39% об’єктів, і серед них є справжні пам’ятки архітектури, яким у OSM просто ніхто не вписав назву.

2. Рейтинг має п’ять сигналів, кожен зведено до частки rank(pct=True): wikidata/wikipedia — 0.35, кількість перекладів назви — 0.3, музей або галерея — 0.15, заповнений historic — 0.1, площа — 0.1. Найбільша вага у сигналів зовнішнього визнання: статтю у Вікіпедії і переклади назви роблять люди, яким об’єкт цікавий, незалежно від OSM. Музей важить більше, ніж просто historic, бо туди можна зайти. Площа найслабша: великим буває і склад. Вікі-статтю мають 47 з 547 кандидатів, переклади назви — 533.

3. У першій десятці збіглося 1 з 10 об’єктів за моїм критерієм і за площею. За площею в десятку потрапили, наприклад, «Ангари» і «Гарматні дворики». Обидва списки зроблено з тих самих даних без помилок. Отже, «найкращі» —

це властивість обраного критерію, а не даних: відповідальність за слово «найкращі» несе аналітик, який обрав ваги.

4. Коефіцієнт обходу: 1.20 для 10 точок і 1.26 для 30. Обидва значення в типовому діапазоні пішохідної мережі 1,2–1,6; для 10 точок — на нижній межі. Причина в місті: історичний центр Одеси — прямокутна сітка кварталів, і пам’ятки десятки стоять уздовж тих самих вулиць і бульварів, тож пішки до них можна йти майже по прямій; на 30 точках з’являються пари через квартали навскіс, і коефіцієнт зростає. Асиметрія матриці — 0 м: на пішохідній мережі немає односторонніх обмежень.

5. Найближчий сусід дав 9.28 км, тобто на 5.4% гірше за точний розв’язок Held–Karp (8.81 км). 2-opt, запущений з маршруту найближчого сусіда, дав 8.81 км, тобто 0.0% від оптимуму — він знайшов саме оптимальний маршрут. На 30 точках найближчий сусід — 28.56 км, 2-opt — 24.02 км (на 15.9% коротше, за 0.006 с). Мене здивувало, що на 30 точках жадібний метод програє вже 19% навіть відносно 2-opt: чим більше точок, тим дорожча «остання точка через усе місто».

6. Таблиця Held–Karp для 31 точки (вокзал і 30 об’єктів) має $2^{30} \cdot 30 \approx 3{,}2 \cdot 10^{10}$ клітин; при 8 байтах на клітину це ≈ 258 ГБ лише для таблиці довжин (і стільки ж для таблиці попередників). Це в десятки разів більше за оперативну пам’ять будь-якого ноутбука, тож точний розв’язок неможливий через брак пам’яті, а не через брак часу.

7. Проблеми в даних і що з ними зроблено:

• без назви — 348 об’єктів; їх відкинуто;

• демонтовані пам’ятники, що досі є в OSM як memorial: Пам'ятник засновникам Одеси (демонтований), Олександр Суворов (демонтований); маршрут вів би до неіснуючої пам’ятки, тому їх прибрано;

• дублікати ближче 60 м: «Одеський художній музей» за 21 м від «Одеський національний художній музей»; «Будинок Лібмана» за 42 м від «Будинок О. П.

Руссова»; «Археологічний музей» за 55 м від «Літературний музей»; «Потьомкінські сходи» за 14 м від «Пам'ятник де Рішельє» — друга точка того самого об’єкта (однаковий wikidata); з кожної пари лишено об’єкт з вищим рейтингом;

• вокзал є і стартом, і пам’яткою («Одеський залізничний вокзал») — у маршруті він зайняв би одне з десяти місць безплатно, тому об’єкти ближче 100 м до старту (Одеський залізничний вокзал — 71 м, EMS Ukraine — 65 м) прибрано з рейтингу; серед них «EMS Ukraine» — будинок із тегом historic=heritage, якому в OSM дали назву кур’єрської служби (опис «Кур'єрські послуги по Україні і всьому світу») — назва об’єкта неправильна;

• у двох пам’ятних гармат «ЗіС-3» тег wikidata вказує на статтю про модель гармати, а не на сам пам’ятник, тож сигнал wiki для них хибний; у топ-30 вони не потрапили, але для рейтингу це урок: зовнішній сигнал теж треба перевіряти;

• об’єкти далі 50 м від пішохідного графа: немає; недосяжних пар у матрицях: 0.

Висновок понад завдання: пам’ятки дуже нерівномірні в просторі — половина з 547 лежить лише в 34 комірках H3 рівня 9 (з 219 зайнятих). Тому маршрут центром короткий, а будь-який «віддалений» об’єкт у десятці різко подовжує його. Живий запит усіх шарів тривав приблизно в 830 разів довше, ніж читання знімка, і повторний запуск не передав мережею 14.5 МБ — для навчального аналізу знімок є правильним проєктним рішенням.

Дані © OpenStreetMap contributors, ліцензія ODbL.

Декларація про використання ШІ: код дописано за допомогою ШІ-помічника Claude (Anthropic) — функція fetch, фільтр кандидатів, рейтинг, чистка дублікатів, check_snap, матриці віддалей, найближчий сусід, 2-opt, H3, sjoin_nearest і карта. Перевірено самостійно: самоперевірки зошита (структура маршрутів, Held–Karp ≤ 2-opt ≤ найближчий сусід); контрольні ознаки методички (є і Point, і Polygon; різних вузлів стільки ж, скільки точок; на другому запуску всі шари — «знімок»); площі й віддалі рахуються в

EPSG:32635; порядок координат перевірено картою (точки в Одесі, а не в океані); функцію пошуку близьких пар перевірено на двох точках за 14 м.

ВИСНОВКИ

Для Одеси дані OpenStreetMap отримано двома способами. Живий запит усіх шарів тривав 310 с і передав 14,5 МБ, а читання збереженого знімка — 0,37 с без жодного трафіку, тож для повторюваного аналізу знімок є правильним проєктним рішенням.

З 895 об'єктів з тегами historic або tourism 348 (39 %) не мають назви, тому кандидатами лишились 547 об'єктів. Критерій «найкращі» побудовано з п'яти сигналів зі спільною шкалою rank(pct=True); з рейтингом за площею він збігся лише в одному об'єкті з десяти, що показує: «найкращі» визначає аналітик, а не дані.

Дані виявились неідеальними: дублікати одного музею й Потьомкінських сходів, демонтовані пам'ятники, вокзал, що водночас є і стартом, і пам'яткою, будинок із назвою кур'єрської служби та wikidata пам'ятних гармат, яка вказує на модель гармати. Усі ці випадки знайдено й оброблено явно.

На пішохідній мережі всі точки сіли на різні вузли, коефіцієнт обходу становить 1,20–1,26. Оптимальний маршрут через 10 пам'яток має довжину 8,81 км; найближчий сусід гірший на 5,35 %, а 2-opt знайшов оптимум. Для 30 об'єктів точний розв'язок потребував би 258 ГБ пам'яті, тому база для ЛР8 — 2-opt з довжиною 24,02 км.

Половина пам'яток міста зосереджена у 34 комірках H3 з 219, тобто в історичному центрі; уздовж маршруту найближчий заклад у середньому за 49 м. Карта маршруту по ребрах графа помітно відрізняється від прямих відрізків: 8,81 км проти 7,13 км.